

CME227 DATA STRUCTURES
Midterm, November, 24, 2013

Number/Name:..... Duration: 90 minutes

1-) (5 pts.) What is a data structure? **A data structure is the physical representation of how data is organized and manipulated.**

2-) (5 pts.) Describe two “real life” applications of a queue and a stack each.

Queue examples: The tennis balls in their container and trays on desk.

Stack examples: bullets in a cartridge and the batteries in flashlight

3-) (10 pts.) What are the advantages (if any) and disadvantages (if any) of a linked list and an array.

Disadvantages of an array:

(1) its size has to be known at compilation

(2) the data in the array are separated in computer memory by the same distance, which means that inserting an item inside the array requires shifting other data in this array

Advantages of an array:

(1) Accessing an element in an array is easy.

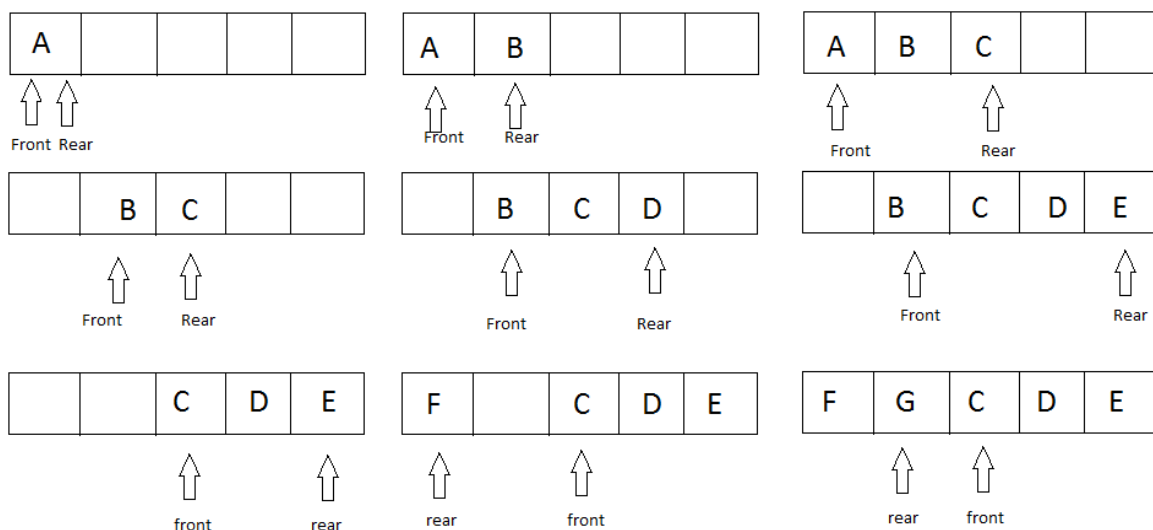
Advantages of a list:

(1) Disadvantages of an array can be overcome by using linked lists.

Disadvantages of an array:

(1) Accessing an element in the list may take some time.

4-) (20 pts.) Draw the circular queue data structure in array implementations for “each step” in the following sequence: **enqueue(A), enqueue (B), enqueue (C), dequeue, enqueue (D), enqueue (E), dequeue, enqueue (F), enqueue (G).** Assume an initial size of 5 for the array implementation. Remember to show both Front and Rear for queue.



5-) (20 pts.) Write a function that makes a copy of a stack. You may declare and use any local variables. You may also use stack operations such as push, pop, top and isempty etc.

Zahidin kodu fena degildi

6-) (22 pts.) Circle T or F for each of the following statements to indicate whether the statement is **true** or **false**, respectively. If the statement is wrong, explain why.

- **T F** Linked lists, stacks and queues are examples of dynamic data structures. **TRUE**
- **T F** Creating and maintaining dynamic data structures requires dynamic memory allocation—the ability for a program to obtain more memory space at execution time to hold new nodes, and to release space no longer needed. **TRUE**
- **T F** A linked list is a linear collection of self-referential structures, called nodes, connected by pointer links—hence, the term "linked" list. **TRUE**
- **T F** Arrays are dynamic, so the length of an array can increase or decrease as necessary. An array is appropriate when the number of data elements to be represented in the data structure is unpredictable. **FALSE**
- **T F** Linked list nodes are normally stored contiguously in memory. **FALSE**
- **T F** A stack is a linked-list that can be accessed from either end. **FALSE**
- **T F** push is used to place elements on the bottom of a stack and pop is used to remove elements from the top of a stack. **FALSE**
- **T F** Queue nodes are removed only from the front (head) of a queue and are inserted only at the back (tail) of the queue. **TRUE**
- **T F** A major advantage of arrays over linked lists is that it takes fewer steps to insert a new element at the end of a long array than at the end of a long list. **TRUE**
- **T F** ADT stands for Abstract Data Tree. **FALSE**
- **T F** A stack, also known as a FIFO buffer, is a first-in-first-out linear data structure. **FALSE**

7-) (20 pts.) Write a method removeRange that takes two integer parameters, min and max, and removes all elements from the list whose values are between min and max, inclusive. It should return the number of elements removed.

Suppose a list stores the following values:

[4, 2, 1, 10, 15, 8, 7, 4, 20, 36, -3, 40, 5]

The call of removeRange(head, 4, 20) would return 8 and change the list to store the following elements:

[2, 1, 36, -3, 40]

If the value of the max parameter is less than that of the min parameter you should throw an `IllegalArgumentException`. If the list is empty or does not contain any elements between or equal to min and max, it should return 0 and leave the list unchanged after a call to `removeRange`.

Do not call any methods of the linked list class to solve this problem.

8-) (20 pts) Fibonacci numbers are sequence of numbers in which first two numbers are $F_0 = 0$, $F_1 = 1$ and other numbers are calculated as follows:

$$F_n = F_{n-1} + F_{n-2};$$

Some fibonacci Numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144 ..

Write a function which generates the first 10 fibonacci number using a doubly circular linked list. You may assume that the first 2 fibonacci numbers are given in the list as follows:

```
struct node *head = malloc(sizeof(struct node));
struct node *temp = malloc(sizeof(struct node));
head->data=0;
temp->data=1;
head->next->=temp;
head->prev->=temp;
temp->next->=head;
temp->prev->=temp;
```